  - For all other processors (Filter, Transform, Tail Sampling, etc.), from the opentelemetry-collector-contrib (http://mng.bz/o1gp) repo
- In the egress path, ingesting signals to the upstream destinations, one or more exporters that can come from two places.
  - For the native OTLP (and the logging exporter), from the opentelemetry-collector (http://mng.bz/nWa2) repo
  - For all other exporters (Prometheus Remote Write, Elasticsearch, AWS X-Ray, etc.), from the opentelemetry-collector-contrib (http://mng.bz/vn04) repo

Let's have a high-level look at some example pipelines from figure 4.6:

- The `pipeline 1` is a traces pipeline that can collect traces via OTLP inbound, batch them up, and ingest them into a Jaeger instance.
- The `pipeline 2` is a metrics pipeline that scrapes metrics via the Prometheus receiver inbound, batches them up again, and uses the Prometheus remote write API to ingest them into, say, Thanos or Cortex.
- The `pipeline N` is a logs pipeline that collects log lines via the Filelog (http://mng.bz/rjJE) receiver inbound, then batches them again and filters them, and can then ingest the log lines into two different backend destinations, Splunk and CloudWatch, via the respective exporters.

In addition, the collector also offers several extensions (https://opentelemetry.io/docs/collector/configuration/#extensions) to operate the agent, including health checks, service discovery, and rich telemetry (e.g., logs, metrics in Prometheus format, traces in OTLP, and profiles in `pprof` format).

> **TIP** There are different ways to deploy (https://opentelemetry.io/docs/collector/deployment/) the collector, usually differentiated between close to the workload (agent mode) and in a central place (gateway mode). Running the collector in gateway mode (effectively, a number of collectors behind a load balancer) can help to enforce policies centrally and support separation of concerns around access control and authentication, including API keys and IAM roles.

Let's have a look at a concrete example that shows the OpenTelemetry collector in action, taking care of both metrics and traces collection and routing; our example setup looks like what is depicted in figure 4.7. We're again using `holly` as the signal generator, exposing metrics in Prometheus exposition format and emitting traces in OTLP format, with the OpenTelemetry collector configured to scrape the metrics and collect the traces and ingest them to respective backend destinations—Prometheus for metrics and Jaeger for traces.

We will now build the overall setup shown in figure 4.7, again using Docker Compose, so let's have a closer look at each of the components in turn. Listing 4.5 shows the sample app `holly`, generating both metrics and traces. Pay special attention to the